

人材マップ（ヒートマップ） 機能仕様

src/pages/HeatmapPage.tsx / src/lib/db/heatmap.ts /
supabase/migrations/20260523_*.sql / supabase/functions/archive-candidates/ の実装を
正とする。

概要

登録済みの人材データを **日本地図上の都道府県別ヒートマップ**として可視化する機能。ナビゲーションタブ
「人材マップ」（5 タブ目）からアクセスする。

- **目的:** 「東京に集中しているのか、地方にも人材がいるのか」を一目で把握する
- **データソース:** candidates（直近7日）+ candidates_archive_light（全期間モード時）
- **集計:** PostgreSQL の RPC で実行（クライアント側集計なし）
- **AI 不使用**

UI 構成（src/pages/HeatmapPage.tsx）

人材分布マップ

[直近7日 / 全期間] [スキルで絞り込み ▼] [クリア]

全人材：N人 X都道府県に分布

日本地図（SVG・d3-geo Mercator）

都道府県を色分け（薄青～濃青）

ホバーでツールチップ

クリックで詳細パネル展開

凡例（少↔多）

都道府県ランキング

Top10 + 棒グラフ

クリックで詳細表示

(クリック時のみ)

受信メール一覧（最大10件・新しい順）

×

日付 名前 件名 [アーカイブ]

主要パーツ

パーツ	内容
期間トグル	直近7日 / 全期間。全期間は candidates_archive_light も合算する
スキルフィルター	テキスト入力でオートコンプリート（skill_master の match_count 降順 200 件）。Enter or 候補クリックで適用、Esc or 「クリア」 ボタンで解除
集計サマリー	「全人材：N 人 X 都道府県に分布」を表示。スキルフィルター適用時は「<スキル> を持つ人材：N 人」に変わる

パーツ	内容
日本地図 SVG	d3-geo の geoMercator() で [136.5, 35.5] を中心に scale 1400 で投影。 public/japan.topojson (約 416KB) を topojson-client で features 化
塗り色	count=0 → 灰 #e5e7eb、それ以上は sqrt(count/max) をベースに薄青 → 濃青のグラデーション。選択中の都道府県は オレンジ #f59e0b
ツールチップ	マウス位置に都道府県名・人数を表示。未選択なら「(クリックで詳細)」も表示
凡例	地図下部にグラデーションバー (少←→多)。動的に maxCount ベースで再生成
ランキング	右ペインに Top10 を棒グラフで表示。クリックで地図と同じ詳細パネルを開く
詳細パネル	都道府県クリック時に画面下部に展開。最大 10 件の受信メールを created_at DESC で表示。アーカイブ済みは「アーカイブ」バッジ付き

キャッシュ戦略 (TanStack Query)

クエリキー	staleTime
['heatmap', dataEnv, skillFilter, period]	60 秒
['skill-names-heatmap']	5 分
['heatmap-candidates', dataEnv, selectedPref, skillFilter, period]	30 秒 (selectedPref 指定時のみ enable)

データ取得層 (src/lib/db/heatmap.ts)

fetchPrefectureCounts(dataEnv, skillFilter, period)

- RPC prefecture_counts(p_data_env, p_skill, p_period) を呼ぶ
- 戻り値: [{ prefecture, count }] (count 降順)

fetchCandidatesByPrefecture(dataEnv, prefecture, skillFilter, period, limit=10)

- RPC candidates_by_prefecture(p_data_env, p_prefecture, p_skill, p_limit, p_period) を呼ぶ
- 戻り値: [{ id, name, subject, created_at, is_archived }] (created_at DESC、最大 limit 件)

fetchSkillNames()

- skill_master テーブルから match_count 降順で 200 件を取得
- オートコンプリート用ドロップダウンの候補に使う

DB 層

新規 migration 4 件

ファイル	内容
<code>20260523_prefecture_counts_rpc.sql</code>	初版 RPC <code>prefecture_counts(text, text)</code> (後段で差し替え)
<code>20260523_archive_light_table.sql</code>	<code>candidates_archive_light</code> テーブル + 期間対応版 RPC <code>prefecture_counts(text, text, text)</code>
<code>20260523_normalize_prefecture.sql</code>	<code>normalize_prefecture(text)</code> 関数 + 最終版 RPC <code>prefecture_counts + candidates_by_prefecture</code>
<code>add_archive_candidates_cron.sql</code>	毎日 JST 9:00 で <code>archive-candidates</code> Edge Function を起動する <code>pg_cron</code> スケジュール。旧 <code>delete-old-candidates</code> を <code>unschedule</code>

テーブル `candidates_archive_light`

7 日経過した人材データを集計用に保持する軽量サマリーテーブル。Storage に置く JSONL より高速に都道府県別集計が取れる。

カラム	型	内容
<code>id</code>	uuid PK	元 <code>candidates.id</code> を保持
<code>data_env</code>	text NOT NULL	'prod' / 'demo'
<code>prefecture</code>	text	元 <code>raw_profile->>'prefecture'</code>
<code>skills</code>	jsonb	スキル配列 (<code>name</code> のフラット)
<code>created_at</code>	timestampz	元 <code>candidates.created_at</code>
<code>archived_at</code>	timestampz	アーカイブ日時
<code>name</code>	text	元 <code>candidates.name</code> (※ <code>archive-candidates</code> Edge Function で upsert される。 <code>candidates_by_prefecture</code> RPC が参照)
<code>subject</code>	text	元 <code>raw_profile->>'subject'</code> (同上)

インデックス:

- `idx_cal_data_env` (`data_env`)
- `idx_cal_prefecture` (`prefecture`)

注意: 現状 `20260523_archive_light_table.sql` は `name / subject` カラムを含めて作成していないが、`archive-candidates/index.ts` と `candidates_by_prefecture` RPC は両カラムの存在を前提に動く。新規環境構築時は `ALTER TABLE candidates_archive_light ADD COLUMN name text, ADD COLUMN subject text;` を追加で実行すること (要修正候補)。

関数 `normalize_prefecture(pref text)`

都道府県名の表記ゆれを吸収して正規化する **IMMUTABLE** 関数。

入力例	出力	ロジック
<code>null / '' / 'null'</code>	NULL	空・無効値
<code>'日本' / '関東' / '全国' / 'リモート'</code>	NULL	国・地方・概念は除外
英字含む (例: <code>'Tokyo'</code>)	NULL	アルファベットを含むものは除外
<code>'東京都 大森'</code>	<code>'東京都'</code>	regex で都道府県接尾辞までを切り出し
<code>'神奈川県'</code>	<code>'神奈川県'</code>	既に正規形
<code>'茨城'</code>	<code>'茨城県'</code>	県名のみで接尾辞なし → 県 を補完 (39 県分の固定リスト)
<code>'東京'</code>	<code>'東京都'</code>	同上 (東京・大阪・京都・北海の特殊 4 件)
その他	NULL	フォールバック

関数 `prefecture_counts(p_data_env, p_skill DEFAULT NULL, p_period DEFAULT '7d')`

```

WITH live AS (
  -- 現行 candidates テーブル (常に対象)
  SELECT DISTINCT c.id, normalize_prefecture(c.raw_profile->>'prefecture')
AS prefecture
FROM candidates c
LEFT JOIN candidate_skills cs ON cs.candidate_id = c.id
WHERE c.data_env = p_data_env
  AND normalize_prefecture(c.raw_profile->>'prefecture') IS NOT NULL
  AND (p_skill IS NULL OR cs.skill ILIKE '%' || p_skill || '%')
),
archived AS (
  -- candidates_archive_light (p_period = 'all' のときのみ追加)
  SELECT DISTINCT a.id, normalize_prefecture(a.prefecture) AS prefecture
FROM candidates_archive_light a
LEFT JOIN LATERAL jsonb_array_elements_text(a.skills) sk
ON p_skill IS NOT NULL
WHERE p_period = 'all'
  AND a.data_env = p_data_env
  AND normalize_prefecture(a.prefecture) IS NOT NULL
  AND (p_skill IS NULL OR sk ILIKE '%' || p_skill || '%')
),
combined AS (
  SELECT id, prefecture FROM live
UNION -- ID 重複を排除 (アーカイブ後も同 ID が残る可能性に対応)
  SELECT id, prefecture FROM archived
)
SELECT prefecture, COUNT(*)::bigint AS cnt

```

```
FROM combined
GROUP BY prefecture
ORDER BY cnt DESC;
```

ポイント:

- スキルフィルターは `candidate_skills.skill ILIKE '%skill%'` (部分一致)
- アーカイブ側は `skills (jsonb)` を `jsonb_array_elements_text` で展開してから部分一致
- `UNION` (`UNION ALL` ではない) で同 ID を排除

関数 `candidates_by_prefecture(p_data_env, p_prefecture, p_skill DEFAULT NULL, p_limit DEFAULT 10, p_period DEFAULT '7d')`

```
SELECT DISTINCT
  c.id, c.name, c.raw_profile->>'subject' AS subject, c.created_at, false
AS is_archived
FROM candidates c
LEFT JOIN candidate_skills cs ON cs.candidate_id = c.id
WHERE c.data_env = p_data_env
  AND normalize_prefecture(c.raw_profile->>'prefecture') = p_prefecture
  AND c.merged_into IS NULL
  AND c.duplicate_flag = false
  AND (p_skill IS NULL OR cs.skill ILIKE '%' || p_skill || '%')

UNION

SELECT DISTINCT
  a.id, a.name, a.subject, a.created_at, true AS is_archived
FROM candidates_archive_light a
LEFT JOIN LATERAL jsonb_array_elements_text(a.skills) sk ON p_skill IS NOT
NULL
WHERE p_period = 'all'
  AND a.data_env = p_data_env
  AND normalize_prefecture(a.prefecture) = p_prefecture
  AND (p_skill IS NULL OR sk ILIKE '%' || p_skill || '%')

ORDER BY created_at DESC
LIMIT p_limit;
```

ポイント:

- `live` 側は `merged_into IS NULL` かつ `duplicate_flag = false` で重複を除外
- アーカイブ側は `is_archived=true` を返す (UI でバッジ表示)
- 最大 `p_limit` 件 (既定 10) を `created_at DESC` で返す

アーカイブ機構 ([supabase/functions/archive-candidates/](https://supabase.com/functions/archive-candidates/))

スケジュール

- 毎日 **JST 9:00 (0:00 UTC)** に pg_cron で起動
- `add_archive_candidates_cron.sql` の `0 15 * * * ...` ※ 実装は **UTC 15:00 = JST 0:00** 起動

処理フロー

- ① 7日以上経過した prod 人材を取得

```
SELECT id, data_env, name, raw_profile, skills, created_at
FROM candidates
WHERE data_env='prod'
      AND created_at < NOW() - INTERVAL '7 days'
ORDER BY created_at ASC
```

↓
- ② candidates_archive_light にサマリー upsert
 - id, data_env, prefecture, skills, created_at, name, subject を保存
 - onConflict: 'id' (再実行に強い)
 ↓
- ③ 関連 submissions を削除

```
DELETE FROM submissions WHERE candidate_id IN (...)
```

↓
- ④ candidates から削除

```
DELETE FROM candidates WHERE id IN (...)
```

旧 delete-old-candidates からの移行

- 旧版は同期で DB を直接削除するだけだった (集計不可能になる問題)
- 新版は **サマリーを残してから削除** することで「全期間ヒートマップ」を可能にした
- マイグレーション `add_archive_candidates_cron.sql` 実行時に旧 cron `delete-old-candidates` を `unschedule` してから新 cron `archive-candidates-daily` を登録 (冪等)

Secrets

`SUPABASE_URL` / `SUPABASE_SERVICE_ROLE_KEY` のみ (自動セット)。

必要なフロントエンド依存

`package.json` に以下を追加:

パッケージ	用途
<code>d3-geo</code>	Mercator 投影 (<code>geoMercator</code> / <code>geoPath</code>)
<code>topojson-client</code>	TopoJSON → GeoJSON 変換 (<code>feature</code>)
<code>@types/topojson-client</code>	型定義
<code>topojson-specification</code>	Topology 型

注: 設計初期に検討した `react-simple-maps` は採用せず、`d3-geo` + 生 SVG `<path>` で実装 (柔軟性とバンドルサイズ重視)。

静的アセット

- **public/japan.topojson** (約 416 KB) : 47 都道府県の geometry。 **properties.nam_ja** (日本語名) と **properties.id** (ID) を保持。Mercator 投影で使用

デプロイ・セットアップ手順

新規環境に展開する場合は以下の順で実施:

1. SQL Editor で migrations を実行

- 20260523_archive_light_table.sql
- 20260523_normalize_prefecture.sql
- 20260523_prefecture_counts_rpc.sql (後段で上書きされるが先に流して OK)
- ALTER TABLE candidates_archive_light ADD COLUMN IF NOT EXISTS name text, ADD COLUMN IF NOT EXISTS subject text; (要追加・現状の migration 漏れ対応)
- add_archive_candidates_cron.sql (YOUR_PROJECT_REF と YOUR_SERVICE_ROLE_KEY を置換)

2. Edge Function archive-candidates をデプロイ

```
supabase functions deploy archive-candidates
```

3. フロントエンドの依存をインストール

```
npm install d3-geo topojson-client topojson-specification @types/topojson-client
```

4. public/japan.topojson を配置 (Git 管理済み)

5. Vercel に再デプロイ

よくある質問・注意点

Q. 「不明」の都道府県は地図に反映される？

A. 反映されません。normalize_prefecture が NULL を返すため prefecture_counts の WHERE で除外されます。'日本' '関東' '全国' 'リモート' 等も同様。

Q. 「東京都 大森」のように住所が長い場合は？

A. normalize_prefecture が '東京都' に正規化して集計します。regex_replace(pref, '^(.+?[都道府県]).*\$', '\\1') で都道府県接尾辞までを切り出します。

Q. 同名駅の同名都道府県問題 (町田・野田 等) には対応している？

A. ヒートマップは `raw_profile-->'prefecture'` を直接参照するため、`STATION_TO_PREFECTURE` の駅マッピングが正しく `prefecture` を埋めていれば問題ありません。`inbound-email` 側の駅マップの精度に依存します。

Q. 「全期間」モードが遅い

A. `candidates_archive_light` のサイズに依存します。インデックス `idx_cal_data_env / idx_cal_prefecture` で都道府県別集計は高速化されていますが、スキルフィルター時は `jsonb` 展開でやや遅くなります (`p_skill IS NOT NULL` のときだけ `LATERAL JOIN`)。

Q. デモ環境のヒートマップは？

A. `dataEnv=demo` で同じ RPC が動きます。デモシードや本番→デモコピーで人材を増やせば即座に地図に反映されます (`staleTime 60` 秒)。

Q. アーカイブされたデータは戻せる？

A. `candidates_archive_light` には集計に必要な最小限 (`prefecture / skills / name / subject` など) しか残らないため、元のフルプロファイルは復元できません。長期保存が必要な場合は Supabase Storage に JSONL でバックアップする運用を別途検討する必要があります。

最終更新: 2026-05-23